

SERVICE ACTUEL · AI-GEN-PROXY

Proxy de modèles et contrôle des fournisseurs

5 capacités analysées

services/ai-gen-proxy · Python/FastAPI, HTTPX, PostgreSQL/Alembic, Kafka, Fernet et JWT.

Acheminer chat et embeddings vers les fournisseurs configurés, administrer leurs clés et observer l'usage.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : AI, Egress, IAM, Audit, Embeddings.

Périmètre de lecture

- Analyses existantes 11-ml-embeddings et 14-front-api reprises comme contexte ; vérifications ciblées des routes, outils et implémentations citées ci-dessous.
- Instructions racine et AGENTS.md du composant lorsqu'il existe ; aucun binaire propriétaire inspecté.
- Lecture statique ciblée pour finalisation ; les branches non exécutées et le déploiement restent non qualifiés.

Objets et cycle de vie

- API keys chiffrées, AIModel, configurations par rôle, métriques et usages.

Dépendances du service

- Fournisseurs HTTP ; IAM/IdP ; PostgreSQL et Kafka.

Fonctionnalités et compatibilité à préserver

AIGP-01

Cible non qualifiée

Administrer les connexions fournisseurs et leurs clés chiffrées

COMPORTEMENT OBSERVÉ

Le plan de contrôle gère création, lecture, liste, mise à jour, suppression et existence des clés fournisseurs. Les secrets sont chiffrés Fernet au repos. La découverte peut réutiliser une clé persistée quand le client fournit un placeholder ou omet le secret. Les routes de gestion valident un JWT et vivent dans une application/port distincts de ceux du proxy.

UTILISATEURS ET RÔLES

Administrateurs de modèles.

INTERFACES DANS LE CODE

/api_keys ; POST /api_keys/discover ; control_app.

DONNÉES ET ÉTATS

API keys, fournisseurs, endpoints et modèles PostgreSQL.

DÉPENDANCES ET ÉCHANGES

IAM, IdP, Fernet, API fournisseur.

COMPORTEMENT CIBLE PROPOSÉ

AI/Egress Rust stocke des références de secrets et autorise chaque endpoint ; séparation réseau contrôle/données conservée.

MIGRATION DE CETTE CAPACITÉ

Migrer les enveloppes via un outil contrôlé puis révoquer les anciens secrets ; conserver les IDs et comparer les droits de gestion.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une clé masquée se modifie sans perte ; le listener de données refuse toutes les routes de gestion ; une destination interdite ne reçoit jamais la clé.

Références de l'analyse

monorepo · services/ai-gen-proxy/app/src/main.py:163 Séparation des applications.

monorepo · services/ai-gen-proxy/app/src/llm_metric/domain/service/discover_models_service.py:45 Réutilisation de connexion existante.

monorepo · services/ai-gen-proxy/AGENTS.md:63 Chiffrement déclaré et contraintes de cache.

AIGP-02 Cible non qualifiée

Acheminer le chat compatible OpenAI, avec streaming et sélection automatique

COMPORTEMENT OBSERVÉ

Le proxy expose le chat par nom technique de clé et une route automatique. Il résout un modèle et sa connexion, déchiffre la clé, transmet la requête et filtre des headers de réponse. Les branches streaming et non-streaming ont leur gestion de délai et métriques. Le streaming demande include_usage et ferme le flux HTTP partagé. Le mode automatique ordonne des modèles par préférence ; son comportement fournisseur doit être qualifié par scénario.

UTILISATEURS ET RÔLES

Applications IA et agents.

INTERFACES DANS LE CODE

POST /llm/{api_key_tech_name}/v1/chat/completions ; POST /llm/v1/chat/completions ; GET /llm/models.

DONNÉES ET ÉTATS

Configuration de modèles ; flux HTTP et métrique de requête.

DÉPENDANCES ET ÉCHANGES

Fournisseurs compatibles, clients HTTP mutualisés, service de métriques.

COMPORTEMENT CIBLE PROPOSÉ

Gateway AI Rust conserve contrat HTTP/SSE, annulation, budgets et choix de modèles autorisés ; Egress borne destinations et tailles.

MIGRATION DE CETTE CAPACITÉ

Rejouer fixtures streaming, erreurs, délais et sélection multi-fournisseur ; basculer par fournisseur.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un client annulé libère le flux ; un timeout retourne une erreur définie ; aucun changement de fournisseur ne viole les politiques de données.

Références de l'analyse

[monorepo](#) · [services/ai-gen-proxy/app/src/llm_metric/application/router/llm_router.py:152](#) Sélection automatique.

[monorepo](#) · [services/ai-gen-proxy/app/src/llm_metric/application/router/llm_router.py:256](#) Flux et collecte de métriques.

AIGP-03 **Cible non qualifiée****Produire les embeddings via les modèles configurés****COMPORTEMENT OBSERVÉ**

L'API liste les modèles d'embedding, expose les informations par clé et transmet une requête d'embeddings au fournisseur sélectionné. Le chemin emploie des retries et enregistre l'usage. Le proxy produit des vecteurs ; il ne publie pas une édition de corpus et ne constitue pas un moteur d'entraînement.

UTILISATEURS ET RÔLES

Ingestions documentaires et applications de recherche.

INTERFACES DANS LE CODE

GET /emb/models ; GET /emb/{api_key_tech_name}/info ; POST /emb/{api_key_tech_name}/v1/embeddings.

DONNÉES ET ÉTATS

Modèles et appels ; vecteurs retournés au consommateur.

DÉPENDANCES ET ÉCHANGES

API fournisseur, clé chiffrée, mesures d'usage.

COMPORTEMENT CIBLE PROPOSÉ

AI sert un contrat modèle/dimension/version ; Embeddings contrôle longueur, type numérique et provenance avant WAP.

MIGRATION DE CETTE CAPACITÉ

Qualifier batch, dimensions, ordre et erreurs de chaque fournisseur ; figer le modèle utilisé dans les éditions.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un batch garde son ordre et sa dimension ; un changement de modèle incompatible est refusé avant publication d'EMBEDDINGS.

Références de l'analyse

[monorepo](#) · [services/ai-gen-proxy/app/src/llm_metric/application/router/embedding_router.py:114](#) Route de vecteurs.

[monorepo](#) · [services/ai-gen-proxy/app/src/llm_metric/application/router/embedding_router.py:201](#) Reprises fournisseur.

AIGP-04 **Cible non qualifiée**

Choisir et sonder les modèles par défaut

COMPORTEMENT OBSERVÉ

La configuration par défaut comprend des rôles de modèles. Pour INSTRUCT et CODER, le service résout les modèles puis lance des probes concurrents avec délai, enregistre les métriques et refuse de sauvegarder si un probe refuse ou lève une erreur. Après succès il upsert les choix puis émet un événement de changement. Les paramètres de génération, dont reasoning_effort, font partie du modèle de configuration.

UTILISATEURS ET RÔLES

Administrateurs et initialisation plateforme.

INTERFACES DANS LE CODE

GET/POST /api_keys/configuration ; ConfigureDefaultModelsService.configure.

DONNÉES ET ÉTATS

DefaultModelConfiguration, rôles, références et résultats de probes.

DÉPENDANCES ET ÉCHANGES

Provider gateway, métriques et événements de configuration.

COMPORTEMENT CIBLE PROPOSÉ

AI Rust versionne le choix par rôle et son attestation de compatibilité ; une probe ne vaut pas qualification complète du modèle.

MIGRATION DE CETTE CAPACITÉ

Reprendre les choix et réglages ; exécuter les probes et tests de capacités avant promotion.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un modèle introuvable ou une probe refusée laisse la configuration antérieure ; tous les replicas observent le changement accepté.

Références de l'analyse

monorepo · services/ai-gen-proxy/app/src/llm_metric/domain/service/configure_default_models_service.py:42 Probes, refus et sauvegarde.

monorepo · services/ai-gen-proxy/app/src/llm_metric/domain/service/configure_default_models_service.py:75 Requête de sondage bornée.

AIGP-05 Cible non qualifiée

Mesurer l'usage, la latence et les erreurs des appels

COMPORTEMENT OBSERVÉ

Le proxy construit une métrique par appel à partir de l'usage fournisseur, du statut, de la durée et du temps au premier token lorsqu'il existe. Il publie les métriques via Kafka et les consomme pour les conserver. Le contrôle expose les modèles de métrique et un résumé de tokens avec filtres. Les écritures en arrière-plan de streaming doivent être distinguées d'une garantie comptable exhaustive.

UTILISATEURS ET RÔLES

Opérateurs et responsables de consommation.

INTERFACES DANS LE CODE

GET /metrics/models ; GET /metrics/tokens ; LLMMetricService.

DONNÉES ET ÉTATS

LLMMetric, usages de tokens, modèle et clé de provenance.

DÉPENDANCES ET ÉCHANGES

Kafka, PostgreSQL et payload d'usage fournisseur.

COMPORTEMENT CIBLE PROPOSÉ

Audit/AI Rust conserve un reçu d'usage corrélé à l'appel et des quotas explicites ; bus sans JVM.

MIGRATION DE CETTE CAPACITÉ

Reprendre métriques et clés ; comparer comptages sur erreurs, annulations et flux tronqués avant bascule.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un même appel rejoué pour transport ne double pas la facturation interne ; un usage absent est inconnu et non assimilé à zéro.

Références de l'analyse

[monorepo](#) · [services/ai-gen-proxy/app/src/llm_metric/application/router/llm_router.py:355](#) Champs de métrique.

[monorepo](#) · [services/ai-gen-proxy/app/src/llm_metric/application/router/llm_metric_router.py:38](#) Résumé des tokens.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- AI/Egress Rust stocke des références de secrets et autorise chaque endpoint ; séparation réseau contrôle/données conservée.
- Gateway AI Rust conserve contrat HTTP/SSE, annulation, budgets et choix de modèles autorisés ; Egress borne destinations et tailles.
- AI sert un contrat modèle/dimension/version ; Embeddings contrôle longueur, type numérique et provenance avant WAP.
- AI Rust versionne le choix par rôle et son attestation de compatibilité ; une probe ne vaut pas qualification complète du modèle.
- Audit/AI Rust conserve un reçu d'usage corrélé à l'appel et des quotas explicites ; bus sans JVM.

Plan de données proposé

- Flux chat et embeddings bornés, sans accès direct au catalogue de publication.

Découpage du core Rust

- ai-gateway, provider-registry, default-models et usage-ledger ; adaptation par fournisseur.

Identité, autorisation et isolation

- Deux listeners ; contrôle exposé par APISIX et JWT obligatoire.
- Secrets chiffrés, aucune valeur secrète incluse dans le dossier.

- Pas de cache local de clés/modèles sans invalidation inter-réplicas selon AGENTS.

Exploitation et reprise

- Clients HTTP partagés fermés par lifespan ; migrations et initialisation au démarrage web.
- Les tests requièrent Docker ; non exécutés.

Séquence de migration

- Migrer les enveloppes via un outil contrôlé puis révoquer les anciens secrets ; conserver les IDs et comparer les droits de gestion.
- Rejouer fixtures streaming, erreurs, délais et sélection multi-fournisseur ; basculer par fournisseur.
- Qualifier batch, dimensions, ordre et erreurs de chaque fournisseur ; figer le modèle utilisé dans les éditions.
- Reprendre les choix et réglages ; exécuter les probes et tests de capacités avant promotion.
- Reprendre métriques et clés ; comparer comptages sur erreurs, annulations et flux tronqués avant bascule.

Écarts et décisions encore ouverts

- Politique d'égress et garanties exactes de retry/fallback à qualifier par fournisseur.
- Le compteur d'usage ne prouve pas à lui seul un quota, un coût contractuel ni une remise exactement une fois.

Références de l'analyse

monorepo · [services/ai-gen-proxy/app/src/main.py:163](#) Séparation des applications.

monorepo · [services/ai-gen-proxy/app/src/llm_metric/application/router/llm_router.py:152](#) Sélection automatique.

monorepo · [services/ai-gen-proxy/app/src/llm_metric/application/router/embedding_router.py:114](#) Route de vecteurs.

monorepo · [services/ai-gen-proxy/app/src/llm_metric/domain/service/configure_default_models_service.py:42](#) Probes, refus et sauvegarde.

monorepo · [services/ai-gen-proxy/app/src/llm_metric/application/router/llm_router.py:355](#) Champs de métrique.